
Learning Conway’s Game of Life with CNN-Transformer Hybrids: Architecture Scaling, Exposure Bias, and Activation Inductive Bias

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We study how a CNN-Transformer hybrid learns the one-step transition rule of
2 Conway’s Game of Life (GoL) on a 40×40 toroidal grid. Starting from a model
3 trained with pure teacher forcing, we show that autoregressive rollout suffers
4 from exposure bias, that a straight-through-estimator (STE) multi-step scheme
5 fails due to a systematic gradient bias, and that scheduled sampling resolves this
6 cleanly. We further show that once a model saturates its capacity, scaling the
7 training set five-fold *regresses* tail-case accuracy in a way invisible to aggregate
8 validation metrics, while scaling the model five-fold instead eliminates essentially
9 all remaining errors. Motivated by recent work on activation-function inductive
10 bias for cellular automata, we independently reproduce the finding that a learnable
11 second-degree polynomial activation lets a 25–34 parameter CNN learn GoL
12 reliably where ReLU fails, and report a negative result transferring this activation
13 to our larger multi-step model. Finally, we measure how much patch embeddings
14 change under rotation, reflection, and translation for two models of different
15 capacity (V8, V10), finding that both are nearly invariant on a sparse glider but that
16 our larger model is roughly twice as sensitive as the smaller one on dense random
17 grids, across every transform tested.

18 1 Problem and architecture

19 GoL is a deterministic cellular automaton: each cell’s next state depends only on its own state and its
20 8-neighbor live count (B3/S23). We treat one GoL step as image-to-image prediction on a periodic
21 40×40 grid. Our core architecture is a CNN-Transformer hybrid: two circular-padded convolutions
22 extract local, translation-equivariant features respecting the torus topology; these are cut into non-
23 overlapping 4×4 patches, linearly projected, combined with a *learned* absolute positional embedding,
24 passed through a Transformer encoder for global mixing, and projected back to per-cell logits. GoL
25 is a natural stress test for learned representations: prior work shows plain feed-forward networks
26 struggle to learn it reliably at all [Springer and Kenyon, 2021], and the Transformer stage [Vaswani
27 et al., 2017] trades the CNN’s clean equivariance guarantee for long-range context (Section 3). This
28 V4 design is itself the fourth step of an earlier architecture search, each step fixing one identified
29 issue: V1 (avg-pooled tokens, learned positional embedding, zero-padding) $\rightarrow$ V2 (swaps in a *fixed*
30 *sinusoidal* positional embedding to remove a right-edge rollout artifact) $\rightarrow$ V3 (swaps avg-pooling
31 for a lossless flatten-and-project tokenization, reverting to a learned embedding) $\rightarrow$ V4 (swaps
32 zero-padding for circular padding, fixing incorrect boundary neighbor counts). V5 onward (Section 2)
33 all train this same V4 architecture under different regimes and sizes.

Ver.	Key change	Val. loss	Prec./Rec.	Status
V1	Baseline: avg-pool tokens, learned PE, zero-pad	— ^b	—	Edge artifacts; rollout freezes to a blob
V2	Learned PE → fixed sinusoidal PE	— ^b	—	Unstable training; no rollout improvement
V3	Avg-pool → flatten+linear tokenization	0.021	—	Fixes avg-pool bottleneck; first realistic rollouts
V4	Zero-padding → circular padding	0.226	100% / 94.3%	Fixes boundary bug; base arch. for V5+
V5	Single-step teacher forcing	0.201	99.4% / 99.7%	Collapses under rollout
V6	+STE multi-step	0.234 ^a	92.7% / —	Diverges
V7	STE → scheduled sampling	0.201	99.7% / 100%	Stable
V8	+3× high-density upsample	0.201	100% / 99.9%	Best per-grid (291K params)
V9	+5× all data	0.201	99.7% / 99.6%	Regressed tail errors
V10	5.15× model size	0.199	100% / 100%	Zero errors , 2,500/2,500

Table 1: Version summary, spanning the V1–V4 architecture search (Section 1) and the V5–V10 training-regime search (Section 2). ^aV6’s best epoch before divergence. ^bV1/V2 reported only aggregate val accuracy (89.4% and 84% respectively), not loss or a precision/recall split.

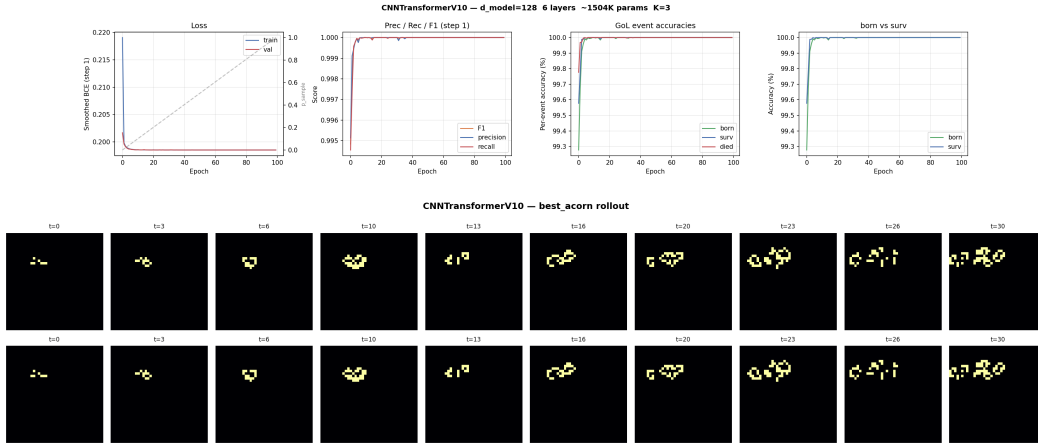


Figure 1: Top: V10 training curve. Bottom: a 30-step autoregressive rollout of the *acorn* methuselah — a 7-cell seed that grows into a large, long-lived chaotic pattern — true trajectory (top row) vs. V10’s autoregressive prediction (bottom row); V10 tracks the true trajectory exactly at every shown step, and makes zero errors across 2,500 held-out grids overall.

2 Training evolution: two failure modes and their fixes

Exposure bias. A model trained with pure single-step teacher forcing (V5) scores > 99% precision/recall on held-out single steps but collapses under autoregressive rollout, since it never sees its own (imperfect) outputs during training. A straight-through-estimator (STE) multi-step scheme intended to fix this instead **diverges** after epoch 5, which we trace to a systematic gradient bias from the STE’s hard threshold. Replacing STE with *scheduled sampling* [Bengio et al., 2015] — gradually increasing the probability of feeding the model its own previous prediction — fixes this cleanly and stably (V7).

A hidden data-scaling regression. Upsampling high-density configurations 3× (V8) improves per-grid accuracy further (291K params). Naively scaling the *training set* 5× (V9) leaves aggregate validation metrics statistically unchanged, but a density-stratified failure search (500 grids at each of five densities, fixed seed across models) reveals V9’s error count at density 0.80 is nearly **seven times worse** than V8’s (22,190 vs. 3,358 summed false positives/negatives) — invisible in aggregate metrics alone. Instead of more data, we scale the *model*: V10 (5.15× V8’s size, 1.5M params, trained from scratch) achieves **zero errors** across all 2,500 test grids at every density (Table 1, Figure 1). The lesson: once capacity saturates, more data can silently trade average-case for tail-case performance, detectable only by per-subgroup error auditing, not validation loss.

51 3 Embedding geometry: does model capacity affect equivariance?

52 A CNN feature map is a *physical* embedding: convolution’s translation-equivariance means shifting
 53 the input shifts the feature map identically. A Transformer’s patch embeddings, after a learned
 54 absolute positional embedding, have no such guarantee — each position slot is an independently
 55 learned vector unrelated to its neighbors, and nothing in the architecture guarantees invariance under
 56 rotation or reflection either. We measure this directly for two stable, well-converged models of
 57 different capacity, V8 (291K params) and V10 (1.5M params), on two base patterns — a sparse glider
 58 and a dense random grid (density 0.5) — under nine individual transforms each: translation by 4 cells
 59 in each of the four cardinal directions, rotation by $90^\circ/180^\circ/270^\circ$, and horizontal/vertical reflection.
 60 For each of the 18 (pattern, transform) cases we measure post-Transformer cosine similarity between
 61 the original grid’s embedding and the transformed grid’s embedding; higher similarity means the
 62 embedding changed less under that specific transform.

63 Breaking this down by individual case (four translation directions, three rotation angles, two reflection
 64 axes) and by pattern type reveals that pattern sparsity, not transform type, is what actually matters
 65 (Figure 2, top). For the sparse *glider*, both models are close to fully invariant to every transform
 66 (0.97–0.99 similarity, V8 and V10 alike) — unsurprising, since a glider occupies only 3 of 100
 67 patches and most patches are unchanged background under any of these transforms. For the dense
 68 *random* grid, the two models diverge sharply: V8 stays around 0.79–0.81 across all nine cases,
 69 while V10 drops to 0.42–0.47 — roughly half. As in Section 2, sparse and dense configurations
 70 behave completely differently, and an evaluation limited to sparse patterns (where both models look
 71 equally equivariant) would have entirely missed V10’s much larger sensitivity on dense grids. This is
 72 consistent with V10 having learned more content-sensitive, less redundant embeddings overall as a
 73 side effect of reaching much higher task accuracy (Section 2), rather than the larger model specifically
 74 losing geometric structure.

75 This measurement also suggests a concrete direction for future work: since GoL’s transition rule
 76 is itself translation- and rotation-invariant, a model whose representations shared that invariance
 77 would have less to learn from data alone. Building on the diagnostic here, one could train with an
 78 explicit consistency loss between a grid’s embedding and its transformed counterpart’s, or design the
 79 positional encoding and attention mechanism to be equivariant by construction, and use the same
 80 per-case measurement to check whether the resulting model both matches this invariance and retains
 81 V10’s dense-grid accuracy.

82 4 Independent verification: polynomial activations for GoL

83 Motivated by a recent claim that a learnable second-degree polynomial activation $\phi(x) = w_0 + w_1x +$
 84 w_2x^2 lets a minimal CNN learn GoL reliably where ReLU fails almost completely [Ahmed and
 85 Davis, 2026], we independently reproduced this rather than taking it on faith. We built two CNNs
 86 matching the paper’s minimal architecture (two 1×1 convolutions plus one 3×3 circular-padded
 87 convolution), **identical except for the activation function**, matching the paper’s reported parameter
 88 counts exactly (ReLU: 25 params, polynomial: 34 params). Trained single-step, teacher-forcing, over
 89 10 random seeds: ReLU converges to perfect validation accuracy in only 1/10 seeds (mean final
 90 accuracy 89.4%), while the polynomial activation converges in 10/10 seeds (100.0% mean accuracy),
 91 confirming the claim on our own toroidal GoL setup (Figure 2, bottom). We then attempted to transfer
 92 this activation into our larger multi-step model (same size as V8) by replacing GELU with polynomial
 93 activations in the CNN stage; this **failed**, diverging sharply after epoch 5 under scheduled sampling,
 94 likely because the unbounded w_2x^2 term compounds through the autoregressive feedback loop in
 95 a way it cannot in a single-step setting. Candidate fixes (a bounded/saturating variant, a separate
 96 learning rate for polynomial coefficients) were identified but not yet pursued.

97 5 Conclusion

98 Three practical lessons emerge. First, autoregressive exposure bias is best fixed with scheduled
 99 sampling rather than a hard differentiable relaxation (STE), which can introduce its own destabilizing
 100 gradient bias. Second, aggregate validation metrics can hide severe, density- specific regressions;
 101 per-subgroup error auditing is necessary once a model is scaled with more data rather than more
 102 capacity. Third, an activation function’s inductive-bias advantage demonstrated at minimal scale and

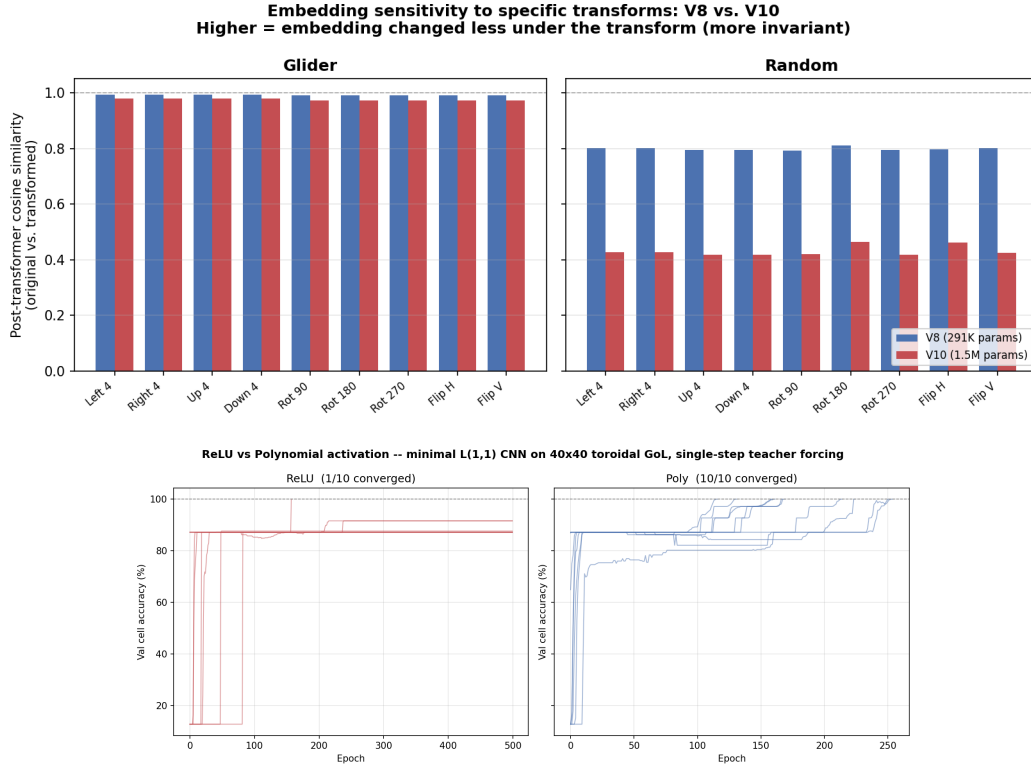


Figure 2: Top: post-Transformer embedding cosine similarity for each of 9 individual transforms (4 translation directions, 3 rotations, 2 reflections), separately for a sparse glider and a dense random grid, V8 vs. V10; higher means the embedding changed less under that transform. Bottom: validation accuracy over training, 10 seeds per activation, on a minimal 25/34-parameter CNN; polynomial activation converges reliably where ReLU does not.

single-step training does not automatically transfer to a larger, multi-step, autoregressively-trained model — the interaction between an activation’s stability properties and the training regime matters as much as the activation itself.

References

- Tashin Ahmed and Q. Tyrell Davis. It’s much easier for neural networks to learn game of life dynamics with the right activation function: Polynomial kolmogorov-arnold networks. *arXiv preprint arXiv:2606.23587*, 2026.
- Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- Jacob M. Springer and Garrett T. Kenyon. It’s hard for neural networks to learn the game of life. *International Joint Conference on Neural Networks (IJCNN)*, 2021.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.